LAPORAN PRAKTIKUM

STRUKTUR DATA

ADT (JAM)

Disusun Oleh:

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

DAPERTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026KATA PENGANTAR

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum Abstract Data Type (ADT) dengan studi kasus Jam ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas praktikum yang telah dilaksanakan.

Praktikum ini bertujuan untuk memahami konsep Abstract Data Type (ADT) melalui implementasi objek Jam dalam bahasa pemrograman Java. Dalam praktikum ini, penulis mempelajari cara merepresentasikan waktu dalam bentuk jam, menit, dan detik, serta mengimplementasikan berbagai operasi seperti validasi waktu, konversi ke detik, penjumlahan waktu, pengurangan waktu, dan perbandingan dua waktu.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Semoga laporan ini dapat bermanfaat bagi pembaca dalam memahami konsep ADT, khususnya pada implementasi objek Jam.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN **2**
2.1 Praktikum ADT "Jam_2511533014" 2
2.2 Praktikum ADT "JamDriver_2511533014 6
2.3 Praktikum ADT "JamDriver2_2511533014" 9
2.4 Tugas Praktikum 12
BAB III PENUTUP **17**
3.1 Kesimpulan 17
DAFTAR PUSTAKA **18**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk mengorganisasi dan menyimpan data agar dapat diakses dan dikelola secara efisien. Salah satu konsep penting dalam struktur data adalah *Abstract Data Type* (ADT), yaitu tipe data yang didefinisikan berdasarkan perilaku (operasi) tanpa memperhatikan implementasi detailnya. ADT "Jam" merupakan representasi waktu yang terdiri dari jam, menit, dan detik. Dalam kehidupan sehari-hari, pengolahan data waktu sangat penting, seperti dalam sistem penjadwalan, aplikasi alarm, dan sistem komputer lainnya.

Dengan menggunakan ADT Jam, kita dapat memahami bagaimana suatu data direpresentasikan serta bagaimana operasi-operasi seperti menambah waktu, mengurangi waktu, dan menampilkan waktu dilakukan secara terstruktur. Praktikum ini membantu mahasiswa memahami konsep ADT secara lebih *konkret* melalui implementasi sederhana.

1.2 Tujuan Praktikum

1. Memahami konsep dasar *Abstract Data Type* (ADT).
2. Mampu merepresentasikan ADT Jam dalam bentuk program.
3. Mengetahui cara melakukan operasi dasar pada ADT Jam (input, output, manipulasi waktu).
4. Melatih kemampuan dalam mengimplementasikan struktur data secara sistematis.

1.3 Manfaat Praktikum

1. Meningkatkan pemahaman mahasiswa tentang konsep *Abstract Data Type* (ADT) melalui praktik langsung.
2. Melatih kemampuan logika dan pemrograman dalam mengolah data waktu (jam, menit, detik).
3. Memberikan gambaran penerapan struktur data dalam pembuatan aplikasi berbasis waktu seperti jadwal dan alarm.

BAB II
PEMBAHASAN

2.1 Praktikum ADT “Jam_2511533014”

```java
package pekan1_2511533014;

public class Jam_2511533014 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    //---validator---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= s && s <= 59);
    }

    //---Konstruktor/Kreator---
    public Jam_2511533014(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss =s;
    }

    //---Selektor---
    public int getHour () { return hh; }
    public int getMinute () { return mm; }
    public int getSecond () { return ss; }

    //---Mutator---
    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m;}
    public void setSecond(int s) {this.ss =s; }

    //---Konversi---
    public int toSecond() { return hh * 3600 + mm * 60  + ss; }
    public static Jam_2511533014 fromSeconds (int total) {
        if (total < 0) throw new IllegalArgumentException ("detik negatif");
```

```java
        total %= 24 * 3600; // wrap 24 jam
        int h= total /3600; total %= 3600;
        int m= total / 60; int s= total % 60;
        return new Jam_2511533014 (h, m, s);

    }

    //---Relasional---
    public int compareTo (Jam_2511533014 other) { return Integer.compare(this.toSecond(), other.toSecond ()); }
    public boolean equals (Object o) {
        if (! (o instanceof Jam_2511533014 j)) return false;
        return hh == j.hh && mm== j.mm && ss== j.ss;

    }
    public int hashCode () { return java.util.Objects.hash(hh, mm, ss); }

    //---Aritmatika---
    public Jam_2511533014 Plus (Jam_2511533014 other) { return fromSeconds (this.toSecond () + other.toSecond()); }
    public Jam_2511533014 minus (Jam_2511533014 other) {return fromSeconds (Math.floorMod(this.toSecond() - other.toSecond(
    public Jam_2511533014 nextSecond() {  return fromSeconds (this.toSecond() +1); }
    public Jam_2511533014 nextNSeconds (int n) { return fromSeconds (this.toSecond() + Math.max(0, n)); }
    public Jam_2511533014 prevSecond() { return fromSeconds (Math.floorMod(this.toSecond()- 1, 24*3600)); }
    public Jam_2511533014 prevNSeconds(int n) { return fromSeconds (Math.floorMod(this.toSecond() -  Math.max(0, n), 24*360

    //---Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
    public static int durasiDetik (Jam_2511533014 jaw, Jam_2511533014 jakh) { return jakh.toSecond() - jaw.toSecond (); }
    // overrriding ke string
    public String toString () { return String.format("%02d:%02d:%02d", hh, mm, ss); }


}
```

Gambar 2.1 Kode Program Praktikum ADT “Jam_2511533014”

Kode tersebut adalah implementasi kelas Java bernama Java Jam_2511533014 yang merepresentasikan waktu dalam format jam, menit, dan

detik. Kelas ini menyediakan atribut untuk menyimpan waktu, method validasi untuk memastikan nilai waktu benar, konstruktor untuk membuat objek, getter dan setter untuk mengakses dan mengubah nilai, serta berbagai operasi seperti konversi ke detik, pembuatan objek dari total detik, perbandingan waktu, operasi aritmatika (penjumlahan, pengurangan, increment/decrement), hingga representasi string. Secara keseluruhan, kelas ini berfungsi sebagai model waktu yang lengkap dengan fitur manipulasi dan validasi.

Berikut penjelasan langkah-langkah kerjanya:

1. package pekan1_2511533014;
   → Mendefinisikan kelas publik bernama Jam_2511533014.
2. public class Jam_2511533014 {
   → Mendefinisikan kelas publik bernama Jam_2511533014.
3. Baris 4-6
   ```
   private int hh; // 0..23
   private int mm; // 0..59
   private int ss; // 0..59
   ```
   → Deklarasi atribut untuk jam, menit, dan detik dengan batas valid
4. Validator baris 9-11
   ```
   public static boolean isValid(int h, int m, int s) {
     return (0 <= h && h <= 23) && (0 <= s && s <= 59);
   }
   ```
   → Method statis untuk mengecek apakah nilai waktu valid (jam 0–23, detik 0–59; *catatan: menit belum dicek, kemungkinan bug*).
5. Konstruktor baris 14-16
   ```
   public Jam_2511533014(int h, int m, int s) {
      this.hh = h; this.mm = m; this.ss = s;
   }
   ```
   → Konstruktor untuk membuat objek dan mengisi nilai jam, menit, detik.
6. Selektor (Getter) baris 19-21
   ```
   public int getHour() { return hh; }
   public int getMinute() { return mm; }
   public int getSecond() { return ss; }
   ```

→ Mengambil nilai jam, menit, dan detik.

7. Baris ke 30-35

```
public static Jam_2511533014 fromSeconds(int total) {
    if (total < 0) throw new IllegalArgumentException("detik negatif");
    total %= 24 * 3600;
    int h = total / 3600; total %= 3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511533014(h, m, s);
}
```

→ Membuat objek waktu dari total detik (dibatasi 24 jam).

8. Relasional baris 39

```
public int compareTo(Jam_2511533014 other) {
    return Integer.compare(this.toSecond(), other.toSecond());
}
```

→ Membandingkan dua waktu berdasarkan detik.

9. Baris 40-43

```
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511533014 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
```

→ Mengecek apakah dua objek waktu sama.

10. Baris 45

```
public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }
```

→ Menghasilkan hash code untuk objek.

11. Aritmatika baris 48

```
public Jam_2511533014 Plus(Jam_2511533014 other) {
    return fromSeconds(this.toSecond() + other.toSecond());
}
```

→ Menjumlahkan dua waktu.

12. Baris 49

```
public Jam_2511533014 minus(Jam_2511533014 other) {
```

```
    return fromSeconds(Math.floorMod(this.toSecond() - other.toSecond(),
24*3600));
}
```

→ Mengurangi waktu dengan hasil tetap dalam 24 jam.

13. Baris 50

```
public Jam_2511533014 nextSecond() {
    return fromSeconds(this.toSecond() + 1);
}
```

→ Menambah 1 detik.

14. Baris 51

```
public Jam_2511533014 nextNSecond(int n) {
    return fromSeconds(this.toSecond() + Math.max(0, n));
}
```

→ Menambah n detik (tidak boleh negatif).

15. Baris 52

```
public Jam_2511533014 prevSecond() {
    return fromSeconds(Math.floorMod(this.toSecond() - 1, 24*3600));
}
```

→ Mengurangi 1 detik.

16. Baris 53

```
public Jam_2511533014 prevNSecond(int n) {
    return fromSeconds(Math.floorMod(this.toSecond() - Math.max(0, n),
24*3600));
}
```

→ Mengurangi n detik.

17. Durasi, baris 56

```
public static int durasiDetik(Jam_2511533014 jaw, Jam_2511533014 jakh)
{
    return jakh.toSecond() - jaw.toSecond();
}
```

→ Menghitung selisih waktu dalam detik.

18. toString, Baris 59

```
public String toString() {
    return String.format("%02d:%02d:%02d", hh, mm, ss);
}
```

→ Mengubah objek menjadi format string HH:MM:SS.

2.2 Praktikum ADT "JamDriver_2511533014

```
1  package pekan1_2511533014;
2
3  public class JamDriver_2511533014 {
4      public static void main(String[] args) {
5          Jam_2511533014 a = new Jam_2511533014 (23, 59, 50);
6          Jam_2511533014 b = new Jam_2511533014(0, 0, 15);
7          System.out.println ("a              = " + a);
8          System.out.println("b              = " + b );
9          System.out.println("a+b            = " + a.Plus(b));
10         System.out.println("next 20s = " + a.nextNSeconds(20));
11         System.out.println("durasi (a,b) = " + Jam_2511533014.durasiDetik (a,b));
12         System.out.println("a.compareTo (b) = " + a.compareTo (b));
13     }
14
15 }
16
```

Gambar 2.2 Kode program praktikum JamDriver

Kode ini adalah kelas JamDriver_2511533014 dalam Java yang berfungsi sebagai program utama (driver) untuk menguji kelas Jam_2511533014. Di dalam method main, dibuat dua objek waktu (a dan b), kemudian dilakukan berbagai operasi seperti menampilkan nilai waktu, menjumlahkan waktu, menambahkan detik, menghitung durasi antara dua waktu, serta membandingkan keduanya. Program ini digunakan untuk memastikan bahwa method-method pada kelas Jam_2511533014 berjalan dengan benar.

Berikut langkah-langkah kerja praktikum:

1. Baris 1

   package pekan1_2511533014;

   → Menentukan package tempat kelas ini berada.

2. Baris 3

   public class JamDriver_2511533014 {

   → Mendefinisikan kelas utama (driver) untuk menjalankan program.

3. Baris 4

   public static void main(String[] args) {

→ Method utama (main) sebagai titik awal eksekusi program.

4. Baris 5

   Jam_2511533014 a = new Jam_2511533014(23, 59, 50);

   → Membuat objek a dengan waktu **23:59:50**.

5. Baris 6

   Jam_2511533014 b = new Jam_2511533014(0, 0, 15);

   → Membuat objek b dengan waktu **00:00:15**.

6. Baris 7

   System.out.println("a = " + a);

   → Menampilkan nilai objek a (otomatis memanggil toString() → format HH:MM:SS).

7. Baris 8

   System.out.println("b = " + b);

   → Menampilkan nilai objek b.

8. Baris 9

   System.out.println("a+b = " + a.Plus(b));

   → Menjumlahkan waktu a dan b menggunakan method Plus(), lalu menampilkan hasilnya.

9. Baris 10

   System.out.println("next 20s = " + a.nextNSeconds(20));

   → Menambahkan **20 detik** ke waktu a, lalu menampilkan hasilnya.

10. Baris 11

    System.out.println("durasi (a,b) = " + Jam_2511533014.durasiDetik(a,b));

    → Menghitung selisih waktu (dalam detik) antara a dan b menggunakan method statis durasiDetik.

11. Baris 12

    System.out.println("a.compareTo (b) = " + a.compareTo(b));

    → Membandingkan waktu a dengan b:

- Nilai negatif → a < b
- Nol → a == b
- Positif → a > b

12. Baris 13-15

    }

    }

    → Penutup method main dan kelas.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.3 dibawah ini.

```
<terminated> JamDriver_2511533014 (1)
a                = 23:59:50
b                 = 00:00:15
a+b               = 00:00:05
next 20s = 00:00:10
durasi (a,b) = -86375
a.compareTo (b) = 1
```

Gambar 2.3 Output dari kode program JamDriver

2.2.1 Analisis

Berdasarkan hasil eksekusi program pada JamDriver, dapat disimpulkan bahwa program telah berhasil mengimplementasikan logika *arithmetic rollover* dalam siklus 24 jam secara akurat. Program terbukti mampu menangani transisi waktu saat melewati batas tengah malam (23:59:59), sehingga operasi penjumlahan $a+b$ maupun fungsi next 20s menghasilkan nilai jam yang tepat pada hari berikutnya. Selain itu, fungsi compareTo dan perhitungan durasi menunjukkan bahwa sistem dapat mengolah perbandingan serta selisih total detik antar objek jam secara konsisten sesuai dengan algoritma yang telah dirancang.

2.3 Praktikum ADT "JamDriver2_2511533014"

```java
package pekan1_2511533014;
import java.util.Scanner;
public class JamDriver2_2511533014 {

        public static void main(String[] args) {
            Scanner input = new Scanner(System.in);
            System.out.println("=== Program Driver Objek Jam ===");

           // 1. Input Jam pertama
            System.out.println("\n--- Input Jam 1 ---");
            Jam_2511533014 j1 = buatJamDariInput(input);

            // 2. Input Jam Kedua
            System.out.println("\n--- Input Jam 2 ---");
            Jam_2511533014 j2 = buatJamDariInput(input);

            // 3. Menampilkan Data
            System.out.println("\n--- Hasil Operasi ---");
            System.out.println("Jam 1 (String)       : " + j1);
            System.out.println("Jam 2 (String)       : " + j2);
            System.out.println("Jam 1 dalam detik    : " + j1.toSecond());
            System.out.println("Jam 2 dalam detik    : " + j2.toSecond());

            // 4. Operasi Relasional
            int perbandingan = j1.compareTo(j2);
            if (perbandingan > 0) {
                System.out.println("Status               : Jam 1 lebih lambat (setelah) Jam 2");
            } else if (perbandingan < 0) {
                System.out.println("Status               : Jam 1 lebih awal (sebelum) Jam 2");
            } else {
                System.out.println("Status               : Jam 1 dan Jam 2 sama");
            }

            // 5. Operasi Aritmatika
            System.out.println("Durasi (j1 ke j2)    : " + Jam_2511533014.durasiDetik(j1, j2) + " detik");
```

```java

            Jam_2511533014 jNext = j1.nextSecond();
            System.out.println("Next Second (j1)     : " + jNext);

            Jam_2511533014 jPrev = j1.prevSecond();
            System.out.println("Prev Second (j1)     : " + jPrev);

            Jam_2511533014 jPlus = j1.Plus(j2);
            System.out.println("j1 + j2              : " + jPlus);

            input.close();
            System.out.println("\nProgram Selesai.");
        }

        // Method bantu input + validasi
        private static Jam_2511533014 buatJamDariInput(Scanner sc) {
            int h, m, s;

            while (true) {
                System.out.print("Masukkan jam (0-23)   : ");
                h = sc.nextInt();
                System.out.print("Masukkan menit (0-59) : ");
                m = sc.nextInt();
                System.out.print("Masukkan detik (0-59) : ");
                s = sc.nextInt();

                if (Jam_2511533014.isValid(h, m, s)) {
                    return new Jam_2511533014(h, m, s);
                } else {
                    System.out.println("[Error] Input tidak valid! Coba lagi.\n");
```

Gambar 2.4 Kode Program praktikum JamDriver2

Kode program tersebut merupakan kelas *driver* atau penguji untuk ADT Jam yang berfungsi mengelola input pengguna, melakukan validasi waktu, serta mendemonstrasikan operasi relasional dan aritmetika antarobjek jam.

Program ini secara sistematis meminta input jam, menit, dan detik, memvalidasinya melalui metode isValid, kemudian menampilkan informasi waktu dalam format string maupun total detik, serta mengeksekusi metode perbandingan (compareTo), perhitungan durasi, dan manipulasi waktu seperti nextSecond, prevSecond, dan Plus.

Berikut langkah-langkah kerjanya:

1. **Baris 1-2:** Deklarasi *package* dan mengimpor *class* Scanner untuk keperluan input data dari *keyboard*.
2. **Baris 3-5:** Deklarasi kelas JamDriver2_2511533014 dan metode utama (main) sebagai titik awal jalannya program.
3. **Baris 6-7:** Inisialisasi objek Scanner dengan nama input dan mencetak judul program ke layar.
4. **Baris 10-11:** Meminta input untuk jam pertama dan memanggil metode buatJamDariInput untuk membuat objek j1.
5. **Baris 14-15:** Meminta input untuk jam kedua dan membuat objek j2 dengan cara yang sama.
6. **Baris 18-22:** Menampilkan data jam yang telah diinput baik dalam bentuk format standar (string) maupun konversi ke total detik menggunakan metode toSecond().
7. **Baris 25:** Melakukan perbandingan antara j1 dan j2 menggunakan metode compareTo dan menyimpan hasilnya dalam variabel perbandingan.
8. **Baris 26-32:** Struktur kontrol if-else untuk menentukan status apakah jam 1 lebih lambat, lebih awal, atau sama dengan jam 2 berdasarkan nilai variabel perbandingan.
9. **Baris 35:** Menghitung dan menampilkan selisih waktu antara jam 1 dan jam 2 dalam satuan detik melalui metode *static* durasiDetik.
10. **Baris 37-44:** Memanggil dan menampilkan hasil manipulasi waktu pada objek j1, yaitu satu detik setelahnya (nextSecond), satu detik sebelumnya (prevSecond), dan hasil penjumlahan antara j1 dan j2 (Plus).

11. **Baris 46-47:** Menutup objek Scanner untuk menghemat sumber daya memori dan menampilkan pesan bahwa program telah berakhir.
12. **Baris 51:** Deklarasi metode pembantu (*helper method*) buatJamDariInput yang bersifat *private* dan *static* untuk menangani proses input secara berulang.
13. **Baris 52-54:** Deklarasi variabel penampung h (jam), m (menit), s (detik) dan memulai perulangan while(true) untuk menjamin input yang masuk harus valid.
14. **Baris 55-60:** Menampilkan instruksi input dan membaca nilai integer untuk jam, menit, serta detik dari pengguna.
15. **Baris 62-63:** Melakukan pengecekan validitas waktu menggunakan metode isValid. Jika valid, objek Jam baru akan dikembalikan (*return*) dan perulangan berhenti.
16. **Baris 64-65:** Jika input tidak valid (misal: menit > 59), program akan menampilkan pesan kesalahan dan meminta pengguna mengulang input dari awal.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.5 dibawah ini.

```
<terminated> JamDriver2_2511533014 (1) [Java Application] C:\Users\rara0\.p2
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan jam (0-23)   : 3
Masukkan menit (0-59) : 46
Masukkan detik (0-59) : 34

--- Input Jam 2 ---
Masukkan jam (0-23)   : 7
Masukkan menit (0-59) : 36
Masukkan detik (0-59) : 21

--- Hasil Operasi ---
Jam 1 (String)       : 03:46:34
Jam 2 (String)       : 07:36:21
Jam 1 dalam detik    : 13594
Jam 2 dalam detik    : 27381
Status               : Jam 1 lebih awal (sebelum) Jam 2
Durasi (j1 ke j2)    : 13787 detik
Next Second (j1)     : 03:46:35
Prev Second (j1)     : 03:46:33
j1 + j2              : 11:22:55

Program Selesai.
```

Gambar 2.5 Output dari kode program JamDriver

2.3.1 Analisis

Pada percobaan ini, program JamDriver2 dijalankan untuk menguji fungsionalitas ADT Jam dalam mengolah input waktu serta melakukan operasi aritmetika dan relasional. Program berhasil memvalidasi input pengguna dan menampilkan informasi waktu baik dalam format string maupun total detik secara akurat.

Dari hasil tersebut dapat disimpulkan bahwa program berjalan dengan benar sesuai dengan logika pemrograman yang diterapkan. Program mampu mengambil keputusan secara otomatis melalui struktur percabangan untuk menentukan status perbandingan antar jam, serta berhasil mengeksekusi manipulasi waktu seperti pencarian detik sebelum, sesudah, dan penjumlahan dua objek jam dengan tepat.

2.4 Tugas Praktikum

2.4.1 Praktikum "Mobil_2511533014"

```java
package pekan1_2511533014;

public class Mobil_2511533014 {

        private String nama;
        private int tahun;
        private int cc;
        private double harga;
        private String merk;

        public Mobil_2511533014(String nama, int tahun, int cc, double harga, String merk) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }

        // Selektor (Getter)
        public String getNama() { return nama; }
        public double getHarga() { return harga; }

        // Mutator (Setter)
        public void setHarga(double harga) { this.harga = harga; }

        public void infoMobil() {
            System.out.println("- " + nama + " (" + merk + "), " + tahun + " [" + cc + "cc] - Rp" + harga);
        }
    }
```

Program ini mengimplementasikan konsep *Encapsulation* dan pengelolaan koleksi objek menggunakan *Array Statis.* Kelas Mobil_2511533014 berfungsi sebagai cetak biru yang menyimpan data teknis mobil dan melindungi atributnya melalui akses *private*, sementara kelas MobilDriver_2511533014 bertindak sebagai pengendali utama. Program ini mampu melakukan manajemen data secara manual, mulai

dari pengisian elemen ke dalam indeks array, pengubahan data menggunakan *mutator*, hingga penghapusan elemen dengan logika pergeseran indeks (*shifting*) untuk menjaga kerapatan data dalam memori.

Berikut langkah-langkah kerjanya:

1. **Baris 1:** Deklarasi package pekan1_2511533014, menandakan bahwa kelas ini berada dalam paket atau folder kerja praktikum pekan pertama.
2. **Baris 3:** Deklarasi kelas public class Mobil_2511533014 sebagai cetakan utama untuk objek mobil.
3. **Baris 5-9:** Deklarasi **Atribut Privat** (nama, tahun, cc, harga, merk). Kata kunci private memastikan variabel ini hanya bisa diakses di dalam kelas ini saja.
4. **Baris 11-16: Constructor**; metode khusus yang otomatis terpanggil saat objek baru dibuat (new) untuk mengisi nilai awal kelima atribut menggunakan parameter yang diterima.
5. **Baris 12-15 (di dalam Constructor):** Penggunaan kata kunci this (seperti this.nama = nama) bertujuan untuk membedakan antara atribut kelas dengan parameter yang namanya sama.
6. **Baris 19-20: Selektor (Getter)**; metode getNama() dan getHarga() digunakan oleh kelas lain (seperti Driver) untuk melihat nilai atribut yang terproteksi.
7. **Baris 23-24: Mutator (Setter)**; metode setHarga() digunakan untuk mengubah nilai harga mobil setelah objek berhasil dibuat.
8. **Baris 26-28:** Metode infoMobil(); sebuah fungsi untuk mencetak ringkasan detail mobil (nama, merk, tahun, cc, dan harga) ke konsol dalam satu format baris yang rapi.

2.4.2 Praktikum "MobilDriver_2511533014"

```java
package pekan1_2511533014;

public class MobilDriver_2511533014 {

        // Array statis dengan kapasitas 100
        private static Mobil_2511533014[] daftarMobil = new Mobil_2511533014[100];
        private static int jumlahMobil = 0; // Penanda indeks

        public static void main(String[] args) {
            System.out.println("=== Program Driver Mobil Array (NIM: 2511533014) ===\n");

            // 1. Tambah Mobil
            tambahMobil(new Mobil_2511533014("Avanza", 2020, 1300, 150000000, "Toyota"));
            tambahMobil(new Mobil_2511533014("Brio", 2021, 1200, 160000000, "Honda"));
            tambahMobil(new Mobil_2511533014("Xpander", 2022, 1500, 250000000, "Mitsubishi"));

            System.out.println("\n--- Daftar Mobil Awal ---");
            tampilkanSemua();

            // 2. Edit Harga (Mutator)
            System.out.println("\n[Sistem] Mengubah harga mobil pertama...");
            daftarMobil[0].setHarga(155000000);

            // 3. Hapus Mobil (Misal hapus indeks ke-1 / Brio)
            hapusMobil(1);

            System.out.println("\n--- Daftar Mobil Akhir ---");
            tampilkanSemua();
        }

        // Method Tambah Mobil ke Array
        public static void tambahMobil(Mobil_2511533014 m) {
            if (jumlahMobil < daftarMobil.length) {
                daftarMobil[jumlahMobil] = m;
                jumlahMobil++;
```

```java
                System.out.println("[Sistem] Berhasil menambah: " + m.getNama());
            } else {
                System.out.println("[Error] Array penuh!");
            }
        }

        // Method Hapus Mobil (Logika Geser Elemen)
        public static void hapusMobil(int index) {
            if (index >= 0 && index < jumlahMobil) {
                System.out.println("[Sistem] Menghapus mobil: " + daftarMobil[index].getNama());

                // Menggeser elemen setelahnya ke kiri agar tidak ada lubang (null)
                for (int i = index; i < jumlahMobil - 1; i++) {
                    daftarMobil[i] = daftarMobil[i + 1];
                }
                daftarMobil[jumlahMobil - 1] = null; // Hapus duplikat di akhir
                jumlahMobil--; // Kurangi counter
            } else {
                System.out.println("[Error] Indeks tidak ditemukan!");
            }
        }

        // Method Selektor untuk menampilkan semua
        public static void tampilkanSemua() {
            if (jumlahMobil == 0) {
                System.out.println("Data kosong.");
            } else {
                for (int i = 0; i < jumlahMobil; i++) {
                    daftarMobil[i].infoMobil();
```

Kelas MobilDriver_2511533014 berfungsi sebagai pengendali utama (*controller*) yang mengatur siklus hidup objek-objek mobil dalam sebuah *array statis.* Program ini mensimulasikan manajemen basis data sederhana dengan menyediakan fitur untuk menambah data ke indeks yang tersedia, mengubah atribut objek melalui *mutator*, serta menghapus data. Poin krusial dalam program ini adalah penggunaan variabel *counter*

jumlahMobil untuk melacak jumlah elemen yang terisi dan penerapan algoritma pergeseran indeks (*shifting*) saat penghapusan dilakukan agar data di dalam array tetap rapat tanpa ada celah kosong (*null*) di tengah.

Berikut langkah-langkah kerjanya:

1. **Baris 1:** Deklarasi package pekan1_2511533014 yang mengelompokkan file ini ke dalam paket praktikum yang sama dengan kelas Mobil.
2. **Baris 3:** Deklarasi kelas MobilDriver_2511533014 sebagai kelas utama yang berisi logika eksekusi.
3. **Baris 6:** Inisialisasi array daftarMobil dengan tipe data Mobil_2511533014 berkapasitas statis sebanyak 100 slot.
4. **Baris 7:** Inisialisasi variabel jumlahMobil bernilai 0 untuk menandai jumlah data yang aktif sekaligus menunjukkan indeks kosong berikutnya.
5. **Baris 9-29:** Metode main yang berisi urutan instruksi program, mulai dari pembuatan objek baru, pemanggilan fungsi tambahMobil, pengubahan harga pada indeks 0 menggunakan setHarga(), hingga proses penghapusan data.
6. **Baris 32:** Deklarasi metode tambahMobil yang menerima parameter objek mobil m.
7. **Baris 33-35:** Pengecekan apakah kapasitas array masih tersedia; jika ya, objek m dimasukkan ke indeks ke-jumlahMobil dan nilai *counter* ditambah satu.
8. **Baris 43:** Deklarasi metode hapusMobil yang menerima parameter index (posisi data yang ingin dibuang).
9. **Baris 44:** Validasi untuk memastikan index yang diminta berada dalam rentang data yang ada (tidak negatif dan tidak melebihi jumlahMobil).
10. **Baris 48-50: Logika Shifting (Penting)**; menggunakan perulangan untuk menimpa elemen pada posisi i dengan elemen di sebelah kanannya (i + 1). Ini dilakukan terus menerus sampai ujung data untuk merapatkan array.

11. **Baris 51-52:** Mengosongkan data terakhir yang kini menjadi duplikat dan mengurangi nilai jumlahMobil karena satu data telah hilang.
12. **Baris 59:** Deklarasi metode tampilkanSemua untuk mencetak seluruh isi array ke layar.
13. **Baris 60-64:** Pengecekan kondisi; jika array tidak kosong, program melakukan iterasi dari indeks 0 hingga jumlahMobil - 1 untuk memanggil metode infoMobil() pada setiap objek.

Dari langkah-langkah kerja kode program diatas maka akan menghasilkan output seperti ini:

```
<terminated> MobilDriver_2511533014 [Java Application] C:\Users\rara0\.
=== Program Driver Mobil Array (NIM: 2511533014) ===

[Sistem] Berhasil menambah: Avanza
[Sistem] Berhasil menambah: Brio
[Sistem] Berhasil menambah: Xpander

--- Daftar Mobil Awal ---
- Avanza (Toyota), 2020 [1300cc] - Rp1.5E8
- Brio (Honda), 2021 [1200cc] - Rp1.6E8
- Xpander (Mitsubishi), 2022 [1500cc] - Rp2.5E8

[Sistem] Mengubah harga mobil pertama...
[Sistem] Menghapus mobil: Brio

--- Daftar Mobil Akhir ---
- Avanza (Toyota), 2020 [1300cc] - Rp1.55E8
- Xpander (Mitsubishi), 2022 [1500cc] - Rp2.5E8
```

Percobaan ini menunjukkan keberhasilan penerapan ADT Mobil menggunakan struktur data *array statis*. Program terbukti mampu menjalankan fungsi manajemen data dasar seperti penambahan objek ke dalam array, pembaruan atribut melalui *mutator*, dan penampilan seluruh informasi secara terstruktur. Selain itu, sistem berhasil menjalankan logika penghapusan data dengan melakukan pergeseran indeks (*shifting*) untuk memastikan tidak ada celah kosong (*null*) di dalam array, sehingga integritas koleksi data tetap terjaga dengan baik.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep Abstract Data Type (ADT) dapat diterapkan dengan baik melalui pembuatan kelas Jam yang merepresentasikan waktu dalam bentuk jam, menit, dan detik. Kelas ini memungkinkan data dan operasi terkait waktu dikemas dalam satu kesatuan yang terstruktur.

Melalui implementasi ini, berbagai operasi seperti validasi waktu, konversi ke detik, penjumlahan, pengurangan, serta perbandingan waktu dapat dilakukan dengan mudah dan sistematis. Selain itu, penggunaan method seperti getter, setter, dan konstruktor membantu dalam pengelolaan data secara lebih aman dan terkontrol.

Dengan demikian, praktikum ini menunjukkan bahwa penerapan ADT dapat meningkatkan keteraturan, modularitas, dan kemudahan dalam pengembangan program. Implementasi objek Jam menjadi contoh nyata bagaimana konsep ADT digunakan untuk menyelesaikan permasalahan dalam pemrograman secara efektif.

DAFTAR PUSTAKA

[1] Oracle. (2026). The Java™ Tutorials – classes and objects. Retrieved April 2, 2026, from https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html. [Diakses: 2-Apr-2026]

[2] Java Code Geeks. (2021). *ADT Java Tutorial – Abstract Data Types in Java*. Retrieved April 2, 2026, from https://examples.javacodegeeks.com/adt-java-tutorial/. [Diakses: 2-Apr-2026]

[3] GeeksforGeeks. (2025). *Abstract Data Types*. Retrieved April 2, 2026, from https://www.geeksforgeeks.org/abstract-data-types/. [Diakses: 2-Apr-2026]